

CellAveragePBE

A Cell-Average Population-Balance Solver for Yeast Fermentation
Codebase Documentation

Kyle G. Hawkins, Ph.D.

github.com.strict015@passmail.net • github.com/Kyle-Hawkins

August 14, 2026

Abstract

CELLAVERAGEPBE is a Julia package that solves a size-structured population balance for a growing, dividing yeast population, coupled to a well-mixed bioreactor chemistry model. The population balance is a mesoscopic integro-partial-differential equation discretized with the Cell Average Technique for division and a first-order upwind flux for growth; the chemistry is a stiff system of eight ordinary differential equations. The fully coupled model reproduces the sustained metabolic oscillations characteristic of the Henson yeast system. This document describes the mathematical model, the numerical methods, the software architecture, and the public API.

The complete source is at <https://github.com/Kyle-Hawkins/CellAveragePBE> (MIT).

Contents

1	Overview	2
2	Mathematical model	2
2.1	Division kernels	2
2.2	Growth kinetics	2
2.3	Bioreactor chemistry	2
3	Numerical methods	3
4	Source files	3
4.1	<code>src/CellAveragePBE.jl</code>	3
4.2	<code>src/parameters.jl</code>	4
4.3	<code>src/grid.jl</code>	6
4.4	<code>src/kernels.jl</code>	7
4.5	<code>src/kinetics.jl</code>	8
4.6	<code>src/cellaverage.jl</code>	9
4.7	<code>src/chemistry.jl</code>	10
4.8	<code>src/model.jl</code>	12
4.9	<code>test/runtests.jl</code>	15
4.10	<code>examples/demo.jl</code>	16
5	Public API	16
6	Usage	17

1 Overview

The package integrates the coupled system

$$\frac{\partial N(x, t)}{\partial t} + \frac{\partial [g N(x, t)]}{\partial x} = \underbrace{B_{\text{div}}(x, t)}_{\text{division birth}} - \underbrace{[S(x) + D] N(x, t)}_{\text{division + dilution loss}}, \quad (1)$$

for the cell number density $N(x, t)$ over cell mass x , together with eight ordinary differential equations for the liquid- and gas-phase chemistry (glucose, ethanol, dissolved and gas-phase oxygen and carbon dioxide, and two “effective” sensed concentrations). Here g is the mass-specific growth rate, $S(x)$ the division (selection) rate, and D the dilution rate. The growth rate and division kernels depend on the local chemistry, closing the coupling.

2 Mathematical model

2.1 Division kernels

A dividing mother of mass y produces daughters distributed according to the breakage kernel

$$b(x, y) = A \left[e^{-\beta(x-m_t)^2} + e^{-\beta(x-(y-m_t))^2} \right], \quad x \leq y, \ y > m_t + m_0, \quad (2)$$

two Gaussian lobes centered on the transition mass m_t and its complement $y - m_t$. The division rate is set by the selection kernel

$$S(x) = \gamma e^{-\varepsilon(x-m_d)^2} \mathbb{K}[m_t - m_0 \leq x \leq m_d] + \gamma \mathbb{K}[x > m_d], \quad (3)$$

a Gaussian rising to a plateau above the division mass m_d . The masses $m_t(S)$ and $m_d(S)$ are piecewise-linear functions of the local substrate concentration $S = G_{\text{eff}} + E_{\text{eff}}$.

2.2 Growth kinetics

The mass-specific growth rate is a sum of three Monod modes (fermentative glucose uptake, oxidative glucose uptake, oxidative ethanol uptake with glucose inhibition):

$$K_{gf}(G) = \mu_{gf} \frac{G}{K_{gf} + G}, \quad K_{go}(G, O) = \mu_{go} \frac{G}{K_{go} + G} \frac{O}{K_{gd} + O}, \quad (4)$$

$$K_{eo}(G, E, O) = \mu_{eo} \frac{E}{K_{eo} + E} \frac{O}{K_{ed} + O} \frac{K_i}{K_i + G}, \quad g = K_{gf} + K_{go} + K_{eo}. \quad (5)$$

All concentration arguments are clamped at zero before entering the saturation terms, which keeps the rates physical (they vanish as a substrate is depleted) and the coupled ODE stable.

2.3 Bioreactor chemistry

The eight chemistry states evolve by feed/dilution, gas-liquid mass transfer, and metabolic demand integrated over the population, e.g. for glucose

$$\frac{dG}{dt} = D(G_f - G) - \int N(x) \left(\frac{K_{gf}}{Y_{gf}} + \frac{K_{go}}{Y_{go}} \right) dx, \quad (6)$$

with analogous balances for ethanol, oxygen, and carbon dioxide. The effective concentrations relax toward the bulk values with a first-order sensing lag, $\dot{G}_{\text{eff}} = \alpha_g(G - G_{\text{eff}})$.

3 Numerical methods

Division (integro-PDE).

The birth term is a double integral over dividing mothers. All newborns produced within a cell are first averaged into a single representative mass $\bar{v} = V_b/B_b$, and that average is then redistributed onto the two neighboring grid nodes — the *Cell Average Technique* of Kumar et al. [1, 2] — which conserves both the number and the mass placed on the coarse grid. Averaging before redistributing is what separates this scheme from the earlier *fixed pivot* technique [3], which splits each birth across its bracketing nodes individually and so discards the information about where inside the cell the births concentrated; on coarse grids that discarding over-predicts the number density in the large-mass tail. The inner breakage integrals are evaluated with adaptive Gauss–Kronrod quadrature (QuadGK), pruned by skipping cells with zero division rate.

Growth (advection).

Cell growth is a first-order *upwind* finite-volume flux $-\partial(gN)/\partial x$ that advects cells up the mass axis.

Coupled integration.

The full state (the n population cells plus eight chemistry ODEs) is advanced with FBDF, a variable-order BDF method, using a finite-difference Jacobian — the non-smooth division kernels are not amenable to automatic differentiation.

Stability.

Clamping the substrate/oxygen concentrations at zero inside the kinetics keeps the fast startup transient physical and the stiff solve well-posed.

4 Source files

The package is organized into small, single-purpose files. The table gives an at-a-glance summary; each file is then described in turn.

File	Responsibility
src/CellAveragePBE.jl	top-level module: dependencies, includes, exports
src/parameters.jl	all physical/numerical constants in one keyword struct
src/grid.jl	mass grid and cell-average interpolation weights
src/kernels.jl	division-mass thresholds, breakage, selection, ethanol formation
src/kinetics.jl	Monod growth kinetics (concentration-clamped)
src/cellaverage.jl	Cell Average Technique birth term
src/chemistry.jl	the eight coupled bioreactor ODEs
src/model.jl	right-hand-side assembly, initial conditions, solve drivers
test/runtests.jl	26-test suite (grid, kernels, kinetics, quadrature, RHS)
examples/demo.jl	minimal runnable demonstration script

4.1 src/CellAveragePBE.jl

The top-level module. It loads the dependencies (QuadGK for quadrature, OrdinaryDiffEq for time integration, and ADTypes for the finite-difference Jacobian specifier), **includes** the component files in dependency order, and **exports** the public API — the parameter, grid, and state types, the kernels and kinetics, and the solve drivers.

```
"""  
    CellAveragePBE
```

A cell-average population-balance solver for yeast fermentation, coupling a size-structured cell population (mesoscopic integro-PDE, discretized by the Cell Average Technique of Kumar et al., 2006) to a well-mixed bioreactor chemistry model (differential-algebraic-flavored ODEs for glucose, ethanol, oxygen, and CO₂). The yeast kinetics and division kernels follow the Henson-group cell population balance model (Zhu et al. 2000; Zhang et al. 2002). See the README

for
full citations.

Quick start:

```
```julia
using CellAveragePBE
out = solve_pbe(n = 60) # build + integrate the coupled system
N, chem = split_state(out.sol.u[end], out.grid.n) # final population + chemistry
```

"""
module CellAveragePBE

using QuadGK: quadgk
using OrdinaryDiffEq
using ADTypes: AutoFiniteDiff

include("parameters.jl")
include("grid.jl")
include("kernels.jl")
include("kinetics.jl")
include("cellaverage.jl")
include("chemistry.jl")
include("model.jl")

export PBEParameters, MassGrid, ChemState
export m_ts, m_ds, breakage, selection, ethanol_formation
export K_gf, K_go, K_eo, K_net
export breakage_births, birth_CA, chemistry_derivs, trapz
export pbe_rhs!, pbe_ode_term, initial_state, solve_pbe, split_state
export pbe_population_rhs!, solve_population

end # module
```

4.2 src/parameters.jl

Defines `PBEParameters`, an immutable `Base.@kwdef` struct holding every physical and numerical constant: the Monod kinetic parameters, the piecewise-linear division-mass constants, the breakage/selection/ethanol kernel shape parameters, the growth regularizers, and the bioreactor-chemistry constants. Collecting all constants in one keyword-constructed struct keeps them out of the function bodies and makes the model reproducible and easy to perturb — any subset is overridden by keyword, e.g. `PBEParameters(D = 0.3)`.

```
"""
    PBEParameters

Physical and numerical constants for the cell-average population-balance model
of
yeast fermentation with Henson-type kinetics. The kinetic, kernel and yield
values
```

are the published parameters of the Henson-group yeast cell population balance model
 (Zhu et al., Chem. Eng. Sci. 55:6155-6167, 2000; Zhang et al., J. Process Control 12:721-734, 2002) -- they were NOT fitted to data here. Collecting every constant in one immutable, keyword-constructed struct keeps them out of the function bodies, which makes the model reproducible and easy to perturb for sensitivity studies: any subset is overridden by keyword, e.g. `PBParameters(D = 0.3)`.

Units: masses in 1e-13 g, concentrations in g/L, rates in 1/h.

"""

Base.@kwdef **struct** PBParameters

--- Monod growth kinetics ---

mu_mgf::Float64 = 30.0 *# max glucose->biomass rate (fermentative)*

mu_mgo::Float64 = 3.25 *# max glucose->biomass rate (oxidative)*

mu_meo::Float64 = 7.0 *# max ethanol->biomass rate (oxidative)*

K_mgf::Float64 = 40.0 *# glucose affinity, fermentative*

K_mgo::Float64 = 2.0 *# glucose affinity, oxidative*

K_meo::Float64 = 1.3 *# ethanol affinity, oxidative*

K_mgd::Float64 = 0.001 *# oxygen affinity, glucose-oxidative*

K_med::Float64 = 0.001 *# oxygen affinity, ethanol-oxidative*

K_inhib::Float64 = 0.4 *# glucose inhibition of ethanol uptake*

--- Division mass thresholds (piecewise-linear in substrate S) ---

S_l::Float64 = 0.1 *# lower substrate breakpoint*

S_h::Float64 = 2.0 *# upper substrate breakpoint*

K_t::Float64 = 0.01 *# slope of transition mass m_t(S)*

m_t0::Float64 = 4.55 *# baseline transition mass*

K_d::Float64 = 3.83 *# slope of division mass m_d(S)*

m_d0::Float64 = 10.25 *# baseline division mass*

m_0::Float64 = 1.0 *# minimum mass increment required to divide*

--- Breakage (daughter-mass) kernel: sum of two Gaussians ---

A::Float64 = sqrt(10 / pi) *# amplitude*

beta::Float64 = 4.0 *# width*

--- Selection (division-rate) kernel ---

gamma::Float64 = 400.0 *# amplitude*

eps::Float64 = 7.0 *# width*

--- Ethanol-formation kernel ---

gamma_e::Float64 = 8.0

eps_e::Float64 = 20.0

me::Float64 = 1.54

--- Growth-birth regularization ---

x0::Float64 = 1e-5

nt0::Float64 = 1.0

--- Bioreactor chemistry ---

G_f::Float64 = 30.0 *# feed glucose*

D::Float64 = 0.15 *# dilution rate*

Y_gf::Float64 = 0.15 *# yield glucose->biomass (fermentative)*

Y_go::Float64 = 0.65 *# yield glucose->biomass (oxidative)*

Y_eo::Float64 = 0.5 *# yield ethanol->biomass (oxidative)*

```

E_f::Float64 = 0.0 # feed ethanol
alpha_g::Float64 = 20.0 # glucose effective-concentration relaxation
alpha_e::Float64 = 20.0 # ethanol effective-concentration relaxation
Kloa::Float64 = 1500.0 # O2 mass-transfer coefficient
H_O::Float64 = 0.032 # O2 Henry constant
F::Float64 = 90.0 # gas flow rate
O_in::Float64 = 0.21 # inlet O2 partial pressure
Vl::Float64 = 0.1 # liquid volume
Vg::Float64 = 0.9 # gas volume
C_in::Float64 = 0.0003 # inlet CO2 partial pressure
Klca::Float64 = 1500.0 # CO2 mass-transfer coefficient
H_C::Float64 = 0.83 # CO2 Henry constant
end

```

4.3 src/grid.jl

Defines `MassGrid`, a uniform mass grid storing the $n + 1$ cell edges (`bounds`) and the n cell midpoints (`centers`); cell i occupies `[bounds[i], bounds[i+1]]`. It also provides two helpers used by the Cell Average Technique: `p_upper`, the upper integration limit for daughters of a given mother cell, and `lambda_weight`, the linear interpolation weight that redistributes a newborn onto a neighboring node.

```

"""
    MassGrid(n; xmax = 12.0)

Uniform mass grid of `n` cells on `[0, xmax]`. Stores the `n+1` cell edges
(`bounds`) and the `n` cell midpoints (`centers`). Cell `i` occupies
`[bounds[i], bounds[i+1]]` with representative mass `centers[i]`.

The grid is uniform in mass. Storing edges and centers separately keeps the
finite-volume bookkeeping explicit: advective fluxes are evaluated on `bounds`,
while populations and kernels are evaluated at `centers`.
"""
struct MassGrid
    n::Int
    bounds::Vector{Float64} # length n+1
    centers::Vector{Float64} # length n
end

function MassGrid(n::Int; xmax::Float64 = 12.0)
    n ≥ 3 || throw(ArgumentError("need at least 3 cells (got $n)"))
    bounds = collect(range(0.0, xmax; length = n + 1))
    centers = [(bounds[i] + bounds[i + 1]) / 2 for i in 1:n]
    return MassGrid(n, bounds, centers)
end

"Upper integration limit for daughters of mother-cell `k` that land in cell `i`
(`i ≤ k`)."
p_upper(grid::MassGrid, i::Int, k::Int) = k == i ? grid.centers[i] : grid.bounds[i + 1]

"""
    lambda_weight(grid, vbar, i, forward)

Linear cell-average interpolation weight used by the Cell Average Technique.
`forward = true` interpolates toward the upper neighbor `i+1`, `false` toward
the lower neighbor `i-1`. Valid for interior cells `2 ≤ i ≤ n-1`.
"""

```

```

"""
function lambda_weight(grid::MassGrid, vbar::Float64, i::Int, forward::Bool)
    xref = forward ? grid.centers[i + 1] : grid.centers[i - 1]
    return (vbar - xref) / (grid.centers[i] - xref)
end

```

4.4 src/kernels.jl

The population-balance kernels, written as plain branchy Julia (replacing the original Heaviside-based lambdas). `m_ts` and `m_ds` give the transition and division masses as continuous piecewise-linear functions of substrate. `breakage` is the two-Gaussian daughter-mass distribution (gated so daughters are lighter than the mother and only sufficiently large mothers divide); `selection` is the division-rate kernel; `ethanol_formation` is the mass-specific ethanol-production weight.

```

# Population-balance kernels: division-mass thresholds, breakage (daughter
# distribution), selection (division rate), and ethanol formation. Each is
# written
# as plain branchy Julia rather than as a product of step functions: the
# branches
# state the gating conditions directly, and they are faster.

"""
    m_ts(S, p)

Transition (minimum-division) mass as a piecewise-linear function of substrate
concentration `S`: constant below `S_l`, linear between `S_l` and `S_h`,
constant
above `S_h`. Continuous at both breakpoints.
"""
function m_ts(S, p::PBParameters)
    if S < p.S_l
        return p.m_t0 + p.K_t * (p.S_l - p.S_h)
    elseif S ≤ p.S_h
        return p.m_t0 + p.K_t * (S - p.S_h)
    else
        return p.m_t0
    end
end

"Mean daughter (division) mass, piecewise-linear in substrate `S`."
function m_ds(S, p::PBParameters)
    if S < p.S_l
        return p.m_d0 + p.K_d * (p.S_l - p.S_h)
    elseif S ≤ p.S_h
        return p.m_d0 + p.K_d * (S - p.S_h)
    else
        return p.m_d0
    end
end

"""
    breakage(x, y, mt, p)

Daughter-mass distribution: probability density that a dividing mother of mass
`y` produces a daughter of mass `x`. Two Gaussian lobes (at `mt` and at the
complementary mass `y - mt`) enforce mass-conserving asymmetric division.
Returns

```

```

0 unless the daughter is lighter than the mother and the mother exceeds the
minimum divisible mass `mt + m_0`.
"""
function breakage(x, y, mt, p::PBParameters)
    (y - x) < 0 && return zero(x) # daughter cannot exceed mother
    (y - mt - p.m_0) ≤ 0 && return zero(x) # mother below minimum divisible mass
    return p.A * exp(-p.beta * (x - mt)^2) + p.A * exp(-p.beta * (x - y + mt)^2)
end

"""
    selection(x, mt, md, p)

Division-rate kernel: rate at which a cell of mass `x` divides. A Gaussian
centered on the division mass `md` (active between `mt - m_0` and `md`) plus a
constant plateau `gamma` for cells heavier than `md`.
"""
function selection(x, mt, md, p::PBParameters)
    term1 = ((x - mt + p.m_0) ≥ 0 && (md - x) ≥ 0) ?
        p.gamma * exp(-p.eps * (x - md)^2) : zero(x)
    term2 = (x - md) > 0 ? p.gamma : zero(x)
    return term1 + term2
end

"""
    ethanol_formation(x, mt, p)

Mass-specific ethanol-formation weight for a cell of mass `x`; a Gaussian offset
above the transition mass `mt`, active only for cells heavier than `mt`.
"""
function ethanol_formation(x, mt, p::PBParameters)
    (x - mt) > 0 || return zero(x)
    return p.gamma_e * exp(-p.eps_e * (x - mt - p.me)^2)
end

```

4.5 src/kinetics.jl

The Monod growth kinetics: K_{gf} , K_{go} , K_{eo} for the three metabolic modes and K_{net} for their sum. Each concentration argument passes through $\text{pos}(x) = \max(x, 0)$ before entering a saturation term, so a transient overshoot to a slightly negative oxygen or glucose concentration cannot make an $O(10^{-3})$ -affinity denominator blow up — the rates simply vanish as a substrate is depleted, keeping the coupled stiff solve well-posed.

```

# Monod-type specific growth kinetics. Arguments are effective concentrations
# (glucose `G`, ethanol `E`, dissolved oxygen `O`). Each rate is defined once,
# at
# top level, so the several call sites that need them cannot drift apart.
#
# Concentrations are clamped at zero (`pos`) before entering the saturation
# terms.
# The oxygen affinities are  $O(1e-3)$ , so a transient overshoot to a slightly
# negative  $O$  would otherwise make  $O/(K+O)$  blow up —clamping keeps the rates
# physical (they vanish as a substrate is depleted) and the coupled ODE stable.

@inline pos(x) = max(x, zero(x))

"Specific fermentative glucose->biomass rate."
K_gf(G, p::PBParameters) = p.mu_mgf * pos(G) / (p.K_mgf + pos(G))

```



```

"Specific oxidative glucose->biomass rate (oxygen-limited)."  

function K_go(G, O, p::PBParameters)  

    g, o = pos(G), pos(O)  

    return p.mu_mgo * g / (p.K_mgo + g) * o / (p.K_mgd + o)  

end  

  

"Specific oxidative ethanol->biomass rate (oxygen-limited, glucose-inhibited)."  

function K_eo(G, E, O, p::PBParameters)  

    g, e, o = pos(G), pos(E), pos(O)  

    return p.mu_meo * e / (p.K_meo + e) * o / (p.K_med + o) * p.K_inhib / (p.  

        K_inhib + g)  

end  

  

"Net specific growth rate, summed over the three metabolic modes."  

K_net(G, E, O, p::PBParameters) = K_gf(G, p) + K_go(G, O, p) + K_eo(G, E, O, p)

```

4.6 src/cellaverage.jl

Implements the Cell Average Technique for the division birth term. **breakage_births** computes, once per right-hand-side evaluation, the number- and mass-birth into every cell as a sum over dividing mother cells (each inner integral by adaptive Gauss–Kronrod quadrature, with zero-rate cells skipped to prune most of the nominal $O(n^2)$ work). **birth_CA** then redistributes those newborns onto each interior node: it forms the average newborn mass $\bar{v} = V/B$ in the cell and its two neighbors and splits each contribution onto the node with linear weights, gated by where $\bar{v}$ falls, so that number and mass are placed consistently on the coarse grid.

```

# Cell Average Technique (Kumar, Peglow, Warnecke, Heinrich & Moerl, Chem. Eng.  

# Sci. 61(10):3327–3342, 2006; extended to growth + breakage in Kumar, Peglow,  

# Warnecke & Heinrich, Powder Technol., 2008) for the breakage + growth birth  

# terms. The key numerical idea: all newborn cells produced anywhere in a bin  

# are  

# first averaged into a single representative mass (vbar = Vb/Bb below), and  

# only  

# then redistributed onto the two neighboring nodes, so that both number and  

# mass  

# are conserved on the coarse grid.  

#  

# This is what distinguishes it from the FIXED PIVOT technique of S. Kumar &  

# Ramkrishna (Chem. Eng. Sci. 51(8):1311–1332, 1996), which splits each birth  

# across its bracketing nodes individually. Averaging first retains information  

# about where inside the cell the births actually concentrated; the fixed pivot  

# discards it and over-predicts the number density in the large-mass tail on  

# coarse grids. That matters here, where n is 60–80 cells.  

  

"""  

    breakage_births(N, S, grid, p) -> (Bb, Vb)  

  

    Number-birth `Bb[i]` and mass-birth `Vb[i]` due to division, for every cell `i`,  

    computed once per right-hand-side evaluation. Each is a sum over mother cells  

    `k ≥ i` of `N[k] · selection(x_k) · [breakage` over the daughter range for cell  

    `i`. Cells with zero selection rate or zero population are skipped, which prunes  

    most of the nominal  $O(n^2)$  work since only heavy cells divide.  

    """  

function breakage_births(N, S, grid::MassGrid, p::PBParameters)  

    mt = m_ts(S, p)  

    md = m_ds(S, p)

```

```

n = grid.n
Bb = zeros(eltype(N), n)
Vb = zeros(eltype(N), n)
for k in 1:n
    Nk = N[k]
    Nk == 0 && continue
    yk = grid.centers[k]
    Sf = selection(yk, mt, md, p)
    Sf == 0 && continue
    w = Nk * Sf
    for i in 1:k
        lo = grid.bounds[i]
        hi = p_upper(grid, i, k)
        hi ≤ lo && continue
        bint, _ = quadgk(x -> breakage(x, yk, mt, p), lo, hi; rtol = 1e-6)
        vint, _ = quadgk(x -> x * breakage(x, yk, mt, p), lo, hi; rtol = 1e-6)
        Bb[i] += w * bint
        Vb[i] += w * vint
    end
end
return Bb, Vb
end

"""
    birth_CA(i, Bb, Vb, grid, p)

Cell-average division birth into interior cell `i` from the precomputed breakage
births (`Bb`, `Vb`). The average newborn mass in each of cells `i-1, i, i+1` is
`vbar = Vb / Bb`; those newborns are redistributed onto node `i` with linear
weights, gated by the location of each average mass relative to the cell nodes,
so that both number and mass are placed consistently on the coarse grid. (Growth
is handled separately as an advection flux; see `pbe_ode_term`.)
"""
function birth_CA(i::Int, Bb, Vb, grid::MassGrid, p::PBEPParameters)
    c = grid.centers
    Bm, Vm = Bb[i - 1], Vb[i - 1]
    Bi, Vi = Bb[i], Vb[i]
    Bp, Vp = Bb[i + 1], Vb[i + 1]

    vbar_m = iszero(Bm) ? zero(Vm) : Vm / Bm # average newborn mass from cell i-1
    vbar_i = iszero(Bi) ? zero(Vi) : Vi / Bi
    vbar_p = iszero(Bp) ? zero(Vp) : Vp / Bp

    T1 = Bm * lambda_weight(grid, vbar_m, i, false) * ((vbar_m - c[i - 1]) > 0)
    T2 = Bi * lambda_weight(grid, vbar_i, i, false) * ((c[i] - vbar_i) > 0)
    T3 = Bi * lambda_weight(grid, vbar_i, i, true) * ((vbar_i - c[i]) > 0)
    T4 = Bp * lambda_weight(grid, vbar_p, i, true) * ((c[i + 1] - vbar_p) > 0)
    return T1 + T2 + T3 + T4
end

```

4.7 src/chemistry.jl

Defines the **ChemState** container for the eight chemistry species, a composite-trapezoid integrator **trapz** for integrating metabolic demand over the population, and **chemistry_derivs**, which returns the eight time derivatives: feed/dilution and gas-liquid mass transfer balanced against the population's substrate uptake and product formation, plus the first-order relaxation of the effective (sensed) concentrations.

```

# Well-mixed bioreactor chemistry: 8 ODEs for liquid- and gas-phase species
# coupled to the cell population through mass-specific uptake/production rates
# integrated over the population  $\int (N(x) \cdot \text{rate}(x) \, dx$  via the trapezoid rule).

"Composite trapezoidal integral of `y` over abscissae `x` (both length `n`)."
function trapz(x::AbstractVector, y::AbstractVector)
    s = zero(eltype(y))
    @inbounds for i in 1:length(x) - 1
        s += (x[i + 1] - x[i]) * (y[i] + y[i + 1]) / 2
    end
    return s
end

"State of the 8 bioreactor chemistry species (liquid G/E, effective G/E, O/CO2
liquid & gas)."
struct ChemState{T}
    G::T; G_eff::T; E::T; E_eff::T; O::T; O_out::T; C::T; C_out::T
end

"""
    chemistry_derivs(N, chem, grid, p) -> NTuple{8}

Time derivatives of the 8 chemistry species. Liquid glucose/ethanol/oxygen/CO2
exchange with feed, gas phase, and the population's metabolic demand; the
"effective" concentrations relax toward the bulk values (a first-order sensing
lag). Returns derivatives in the order
`(G, G_eff, E, E_eff, O, O_out, C, C_out)`.
"""
function chemistry_derivs(N, chem::ChemState, grid::MassGrid, p::PBEPParameters)
    G, G_eff, E, E_eff = chem.G, chem.G_eff, chem.E, chem.E_eff
    O, O_out, C, C_out = chem.O, chem.O_out, chem.C, chem.C_out
    x = grid.centers
    S = G_eff + E_eff
    mt = m_ts(S, p)

    kgf = K_gf(G_eff, p)
    kgo = K_go(G_eff, O, p)
    keo = K_eo(G_eff, E_eff, O, p)
    fE = ethanol_formation.(x, mt, Ref(p)) # mass-specific ethanol weight per
    cell

    # Glucose: feed/dilution minus fermentative + oxidative uptake
    dG = p.D * (p.G_f - G) - trapz(x, N .* (kgf / p.Y_gf + kgo / p.Y_go))
    # Ethanol: dilution + fermentative production minus oxidative consumption
    dE = p.D * (p.E_f - E) + trapz(x, N .* (92 / 180 .* fE .* kgf / p.Y_gf .- keo
    / p.Y_eo))
    # First-order effective-concentration relaxation
    dG_eff = p.alpha_g * (G - G_eff)
    dE_eff = p.alpha_e * (E - E_eff)
    # Dissolved O2: gas-liquid transfer minus respiratory demand
    dO = p.Kloa * (p.H_O * O_out - O) - trapz(x, N .* (96 / 46 * keo / p.Y_eo +
    192 / 180 * kgo / p.Y_go))
    dO_out = (p.F * (p.O_in - O_out) - p.Kloa * (p.H_O * O_out - O) * p.Vl) / p.
    Vg
    # Dissolved CO2: gas-liquid transfer plus respiratory + fermentative
    production
    dC = p.Klca * (p.H_C * C_out - C) +

```

```

        trapz(x, N .* (264 / 180 * kgo / p.Y_go + 88 / 46 * keo / p.Y_eo .+ 88 /
            180 .* fE .* kgf / p.Y_gf))
    dC_out = (p.F * (p.C_in - C_out) - p.Klca * (p.H_C * C_out - C) * p.Vl) / p.
        Vg

    return (dG, dG_eff, dE, dE_eff, dO, dO_out, dC, dC_out)
end

```

4.8 src/model.jl

Assembles and solves the coupled system. `pbe_ode_term` gives the net rate for one interior cell — division birth (from `cellaverage.jl`) plus the first-order upwind growth-advection flux, minus dilution and division loss. `pbe_rhs!` is the in-place right-hand side of the full $n + 8$ state (population cells with fixed end caps, followed by the chemistry). `initial_state` builds the bimodal initial distribution and chemistry vector; `solve_pbe` constructs the `ODEProblem` and integrates it with a stiff BDF solver (the `N_scale` keyword sets the cell-number-to-concentration coupling factor). `pbe_population_rhs!` and `solve_population` provide the population-only variant in a fixed chemical environment, and `split_state` unpacks a solution vector into $(N, \text{ChemState})$.

```

# Assembly of the full coupled system: n population-balance ODEs (one per mass
# cell) coupled to the 8 chemistry ODEs, plus initial conditions and a solve
# driver. State layout: u[1:n] = number density N, u[n+1:n+8] = chemistry.

"""
    pbe_ode_term(i, N, chem, Bb, Vb, grid, p)

Net rate of change of the number density in interior cell `i`:

* **division birth** -cell-average redistribution of breakage products,
* **growth advection** -first-order upwind flux  $\partial^-(g \cdot N) \partial / x$  that moves cells
  up the mass axis at the mass-specific growth rate  $g = K_{\text{net}}$  (uniform in
  size, so the flux is  $-(g \cdot N_i - g \cdot N_{i-1}) \Delta / x$ ),
* **death** -loss to dilution  $D$  and to division  $\text{selection}(x_i)$ .

Growth is now a genuine advection term rather than the self-canceling birth/
death
of the original formulation, so cells grow toward the division mass and sustain
the population.
"""
function pbe_ode_term(i::Int, N, chem::ChemState, Bb, Vb, grid::MassGrid, p::
    PBEParameters)
    S = chem.G_eff + chem.E_eff
    g = K_net(chem.G_eff, chem.E_eff, chem.O, p)
    dx = grid.bounds[i + 1] - grid.bounds[i]
    mt = m_ts(S, p); md = m_ds(S, p)

    birth = birth_CA(i, Bb, Vb, grid, p)
    advection = -g * (N[i] - N[i - 1]) / dx # upwind (g ≥ 0)
    death = (p.D + selection(grid.centers[i], mt, md, p)) * N[i]
    return birth + advection - death
end

"""
    pbe_rhs!(du, u, params, t)

In-place right-hand side of the coupled PBE + chemistry system. `params` is the

```

```

tuple `(grid, p)`. Boundary cells `1` and `n` are held fixed (no-flux end caps),
matching the original formulation.
"""
function pbe_rhs!(du, u, params, t)
    grid, p = params
    n = grid.n
    N = @view u[1:n]
    chem = ChemState(u[n + 1], u[n + 2], u[n + 3], u[n + 4],
                    u[n + 5], u[n + 6], u[n + 7], u[n + 8])

    S = chem.G_eff + chem.E_eff
    Bb, Vb = breakage_births(N, S, grid, p) # once per evaluation, shared by all
    cells

    du[1] = 0.0
    du[n] = 0.0
    @inbounds for i in 2:n - 1
        du[i] = pbe_ode_term(i, N, chem, Bb, Vb, grid, p)
    end

    dchem = chemistry_derivs(N, chem, grid, p)
    @inbounds for j in 1:8
        du[n + j] = dchem[j]
    end
    return nothing
end

"""
    initial_state(grid, p; kwargs...)

Bimodal initial cell distribution (two Gaussian sub-populations) stacked with
the
initial chemistry vector `(G, G_eff, E, E_eff, O, O_out, C, C_out)`.
"""
function initial_state(grid::MassGrid, p::PBEPParameters;
                      center1 = 4.0, center2 = 7.5, sigma = 1.0, N1 = 350.0, N2 =
                      450.0,
                      chem0 = (4.0, 3.0, 2.0, 1.0, 0.1, 0.2, 0.05, 0.1))
    x = grid.centers
    N0 = @. N1 * exp(-(center1 - x)^2 / sigma^2) + N2 * exp(-(center2 - x)^2 /
    sigma^2)
    return vcat(N0, collect(float.(chem0)))
end

"""
    solve_pbe(; n, tspan, saveat, p, solver, kwargs...)

Build and solve the coupled model. Returns a named tuple `(sol, grid, p)` where
`sol` is the `OrdinaryDiffEq` solution (state length `n + 8`). A stiff solver is
used by default because the gas-liquid mass-transfer terms are stiff.
"""
function solve_pbe(; n::Int = 60, tspan = (0.0, 20.0),
                  saveat = range(tspan[1], tspan[2]; length = 200),
                  p::PBEPParameters = PBEPParameters(), N_scale = 1.0,
                  solver = FBDF(autodiff = AutoFiniteDiff()), reltol = 1e-6, abstol
                  = 1e-8, kwargs...)
    grid = MassGrid(n)
    u0 = initial_state(grid, p)

```

```

    u0[1:n] .*= N_scale # scale population to the chemistry (cell-number-to-
        concentration factor)
    prob = ODEProblem(pbe_rhs!, u0, tspan, (grid, p))
    sol = solve(prob, solver; saveat = saveat, reltol = reltol, abstol = abstol,
        kwargs...)
    return (sol = sol, grid = grid, p = p)
end

"""
    pbe_population_rhs!(du, N, params, t)

Right-hand side of the population balance alone, evolving the size distribution
`N` in a fixed chemical environment `env::ChemState`. `params = (grid, p, env)`
.

Isolating the population from the (stiff, tightly coupled) chemistry gives a
clean,
well-posed demonstration of the Cell Average Technique -division redistributes
newborns while growth advects mass up the grid.
"""
function pbe_population_rhs!(du, N, params, t)
    grid, p, env = params
    n = grid.n
    S = env.G_eff + env.E_eff
    Bb, Vb = breakage_births(N, S, grid, p)
    du[1] = 0.0
    du[n] = 0.0
    @inbounds for i in 2:n - 1
        du[i] = pbe_ode_term(i, N, env, Bb, Vb, grid, p)
    end
    return nothing
end

"""
    solve_population(; n, tspan, saveat, env, p, solver, kwargs...)

Integrate the population balance in a fixed chemical environment `env`. Returns
`(sol, grid, p, env)`; each `sol.u` is the length-`n` number density `N(t)`.
"""
function solve_population(; n::Int = 60, tspan = (0.0, 4.0),
    saveat = range(tspan[1], tspan[2]; length = 100),
    env::ChemState = ChemState(5.0, 5.0, 0.5, 0.5, 0.05, 0.0,
        0.0, 0.0),
    p::PBParameters = PBParameters(),
    N0::Union{Nothing, AbstractVector} = nothing,
    solver = FBDF(autodiff = AutoFiniteDiff()),
    reltol = 1e-6, abstol = 1e-8, kwargs...)
    grid = MassGrid(n)
    u0 = N0 === nothing ? initial_state(grid, p)[1:n] : collect(float.(N0))
    length(u0) == n || throw(ArgumentError("N0 must have length n = $n"))
    prob = ODEProblem(pbe_population_rhs!, u0, tspan, (grid, p, env))
    sol = solve(prob, solver; saveat = saveat, reltol = reltol, abstol = abstol,
        kwargs...)
    return (sol = sol, grid = grid, p = p, env = env)
end

"Split a solution state vector into `(N, ChemState)` for cell count `n`."
function split_state(u, n::Int)
    N = u[1:n]

```

```

    chem = ChemState(u[n + 1], u[n + 2], u[n + 3], u[n + 4],
                    u[n + 5], u[n + 6], u[n + 7], u[n + 8])
    return N, chem
end

```

4.9 test/runtests.jl

The 26-test suite. It checks grid construction and the edge/center invariants; continuity of the division-mass functions at their breakpoints and the gating of the breakage, selection, and ethanol-formation kernels; saturation and positivity of the Monod kinetics; the trapezoidal integrator against known integrals; and that the assembled right-hand side is finite, correctly sized, and holds the boundary cells fixed.

```

using CellAveragePBE
using Test

const p = PBEParameters()

@testset "MassGrid" begin
    g = MassGrid(60; xmax = 12.0)
    @test length(g.bounds) == g.n + 1
    @test length(g.centers) == g.n
    @test g.bounds[1] == 0.0 && g.bounds[end] ≈ 12.0
    # centers strictly inside their cells, monotone increasing
    @test all(g.bounds[i] < g.centers[i] < g.bounds[i + 1] for i in 1:g.n)
    @test issorted(g.centers)
    @test_throws ArgumentError MassGrid(2)
end

@testset "Kernel continuity & gating" begin
    # m_ts / m_ds continuous at the two substrate breakpoints
    for m in (m_ts, m_ds)
        @test m(p.S_l - 1e-8, p) ≈ m(p.S_l + 1e-8, p) atol = 1e-6
        @test m(p.S_h - 1e-8, p) ≈ m(p.S_h + 1e-8, p) atol = 1e-6
    end
    mt = m_ts(1.0, p)
    # breakage vanishes when daughter exceeds mother or mother too small to divide
    @test breakage(9.0, 6.0, mt, p) == 0.0 # daughter > mother
    @test breakage(1.0, mt + 0.5 * p.m_0, mt, p) == 0.0 # mother below mt + m_0
    @test breakage(mt, 12.0, mt, p) > 0.0 # peak near transition mass
    # selection is nonnegative and plateaus above the division mass
    md = m_ds(1.0, p)
    @test selection(md + 1.0, mt, md, p) ≈ p.gamma
    @test selection(0.0, mt, md, p) == 0.0
    # ethanol formation gated above transition mass
    @test ethanol_formation(mt - 1.0, mt, p) == 0.0
    @test ethanol_formation(mt + p.me, mt, p) > 0.0
end

@testset "Monod kinetics" begin
    @test K_gf(0.0, p) == 0.0
    @test K_gf(1e6, p) ≈ p.mu_mgf rtol = 1e-3 # saturates at mu_mgf
    @test K_net(4.0, 2.0, 0.1, p) ≈
        K_gf(4.0, p) + K_go(4.0, 0.1, p) + K_eo(4.0, 2.0, 0.1, p)
    @test K_net(4.0, 2.0, 0.1, p) > 0.0
end

```

```

@testset "Trapezoid rule" begin
    x = collect(range(0, 1; length = 101))
    @test trapz(x, x) ≈ 0.5 rtol = 1e-6 #  $\int_0^1 x \, dx$ 
    @test trapz(x, x.^2) ≈ 1/3 rtol = 1e-3 #  $\int_0^1 x^2 \, dx$ 
end

@testset "RHS well-formed" begin
    g = MassGrid(40)
    u0 = initial_state(g, p)
    du = similar(u0)
    pbe_rhs!(du, u0, (g, p), 0.0)
    @test all(isfinite, du)
    @test du[1] == 0.0 && du[g.n] == 0.0 # boundary cells held fixed
    @test length(du) == g.n + 8
end

```

4.10 examples/demo.jl

A minimal, dependency-light script that solves the coupled model and the population-only model and prints summary diagnostics — a quick check that the package runs end to end.

```

# Minimal runnable demo of CellAveragePBE (no plotting dependencies).
# Run: julia --project=. examples/demo.jl

using CellAveragePBE

println("Coupled bioreactor -- metabolic oscillations")
out = solve_pbe(n = 60, tspan = (0.0, 25.0), p = PBEParameters(D = 0.3), N_scale = 1/500)
sol = out.sol; n = out.grid.n
G = [split_state(u, n)[2].G for u in sol.u]
E = [split_state(u, n)[2].E for u in sol.u]
println(" retcode = ", sol.retcode)
println(" glucose range = ", round(minimum(G), sigdigits = 3), " .. ",
        round(maximum(G), sigdigits = 3), " g/L")
println(" ethanol maximum = ", round(maximum(E), sigdigits = 3), " g/L")

println("\nPopulation balance alone (fixed environment)")
outp = solve_population(n = 80, tspan = (0.0, 4.0))
println(" retcode = ", outp.sol.retcode)
println(" final cell total = ", round(sum(outp.sol.u[end]), sigdigits = 4))

```

5 Public API

Types.

PBEParameters

Immutable, keyword-constructed struct holding every model constant (kinetics, division thresholds, kernels, chemistry). Override any subset by keyword, e.g. `PBEParameters(D = 0.3)`.

MassGrid(n; xmax)

A uniform mass grid of `n` cells on `[0, xmax]`, storing cell edges and midpoints.

ChemState

The eight chemistry states ($G, G_{\text{eff}}, E, E_{\text{eff}}, O, O_{\text{out}}, C, C_{\text{out}}$).

Drivers.

`solve_pbe(; n, tspan, p, N_scale, ...)`

Build and integrate the fully coupled population + chemistry system. Returns (`sol`, `grid`, `p`); each solution state has length $n + 8$. `N_scale` sets the cell-number-to-concentration coupling factor.

`solve_population(; n, tspan, env, N0, ...)`

Integrate the population balance alone in a fixed chemical environment `env`; useful for isolating the growth/division numerics.

`split_state(u, n)`

Split a solution state vector into (`N`, `ChemState`).

Kernels and kinetics. `m_ts`, `m_ds`, `breakage`, `selection`, `ethanol_formation`, `K_gf`, `K_go`, `K_eo`, `K_net` are all exported for inspection and plotting.

6 Usage

```
using Pkg; Pkg.activate("."); Pkg.instantiate()
using CellAveragePBE

# Coupled bioreactor -- sustained metabolic oscillations
out = solve_pbe(n = 60, tspan = (0.0, 25.0), p = PBEParameters(D = 0.3), N_scale
    = 1/500)
N, chem = split_state(out.sol.u[end], out.grid.n)

# Population balance alone, in a fixed chemical environment
outp = solve_population(n = 80, tspan = (0.0, 4.0))
```

7 Testing

The suite in `test/runtests.jl` (26 tests) verifies grid construction, continuity and gating of the kernels, saturation of the Monod kinetics, the trapezoidal integrator, and well-formedness of the right-hand side. Run it with:

```
julia --project=. -e 'using Pkg; Pkg.test()'
```

References

- [1] Jitendra Kumar, Mirko Peglow, Gerald Warnecke, Stefan Heinrich, and Lothar Mörl. Improved accuracy and convergence of discretized population balance for aggregation: the cell average technique. *Chemical Engineering Science*, 61(10):3327–3342, 2006. The cell average technique itself. This is the scheme implemented in `src/cellaverage.jl`.
- [2] Jitendra Kumar, Mirko Peglow, Gerald Warnecke, and Stefan Heinrich. An efficient numerical technique for solving population balance equation involving aggregation, breakage, growth and nucleation. *Powder Technology*, 2008. Cell average extended to combined growth and breakage, which is the combination solved here. Volume and page numbers not verified against the publisher record.

- [3] Sanjeev Kumar and Doraiswami Ramkrishna. On the solution of population balance equations by discretization—I. A fixed pivot technique. *Chemical Engineering Science*, 51(8):1311–1332, 1996. The fixed pivot technique – the sectional lineage that the cell average method improves on. Not the method implemented here.
- [4] Doraiswami Ramkrishna. *Population Balances: Theory and Applications to Particulate Systems in Engineering*. Academic Press, San Diego, 2000. General reference for the population balance formulation.
- [5] Guang-Yan Zhu, Abdelqader Zamamiri, Michael A. Henson, and Martin A. Hjortsø. Model predictive control of continuous yeast bioreactors using cell population balance models. *Chemical Engineering Science*, 55:6155–6167, 2000. Source of the yeast kinetics, division kernels and parameter values used here.
- [6] Yongchun Zhang, Abdelqader M. Zamamiri, Michael A. Henson, and Martin A. Hjortsø. Cell population models for bifurcation analysis and nonlinear control of continuous yeast bioreactors. *Journal of Process Control*, 12:721–734, 2002. Companion cell population balance model; the sustained oscillations reproduced here are this model class.
- [7] Michael A. Henson. Dynamic modeling of microbial cell populations. *Current Opinion in Biotechnology*, 2003. Review of this model class. Volume and page numbers not verified against the publisher record.

The division kernels, the piecewise-linear transition and division masses, the three-mode Monod kinetics and the effective (sensed) concentrations follow the Henson-group yeast model [5, 6, 7]. The parameters in `src/parameters.jl` are that model’s published values; they were not fitted here.